6

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Container Güvenliği & İmaj Optimizasyonu: Distroless, Alpine & Non-Root

Container Security & Image Optimization: Distroless, Alpine & Non-Root

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Uçağa binerken yanınıza 150 kiloluk gardırop almazsınız; hem fazla para ödersiniz hem de hırsızlara davetiye çıkarırsınız! Standart Ubuntu imajında hacker'ların kullanabileceği 500 tane gereksiz araç (curl, apt, bash) vardır. Zırhlı mini imajda (Alpine veya Distroless) ise sadece Java çalışır. Üstüne bir de kapıya 'ROOT GİREMEZ' tabelası asarız!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't travel with a 150kg solid oak wardrobe on an airplane! Bloated images carry 500 unused utilities (bash, curl, package managers) that attackers love to exploit. Hardened minimal images (Alpine, Distroless) contain ONLY your Java binary. And we lock out the superuser by hanging a strict 'NO ROOT ALLOWED' sign!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Varsayılan olarak Docker konteynerleri `root` (UID 0) kullanıcısıyla çalışır. Konteynerden kaçış (container breakout) açığı oluşursa, saldırgan ana sunucuda da tam yetkili root olur! Bu yüzden: 1) Dockerfile içinde özel bir `appuser` (UID 10001) tanımlanmalıdır. 2) Paket yöneticisi ve kabuk (shell) bile içermeyen Google Distroless veya hafif Alpine imajları tercih edilmelidir. 3) Trivy veya Grype ile imajdaki CVE güvenlik açıkları CI pipeline'ında taranmalıdır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

By default, containers execute as `root` (UID 0). In the event of a Linux kernel or container breakout vulnerability, an attacker gains root on the physical host. Mitigation requires: 1) Enforcing non-root users (`USER 10001`), 2) Using shell-less Distroless bases, 3) Automated scanning for known CVEs via Trivy.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Güvenli Dockerfile Snippet:
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# 1. Root olmayan grup ve kullanıcı yarat
RUN addgroup -S appgroup -g 10001 && \
    adduser -S appuser -u 10001 -G appgroup

# 2. Sahipliği ver ve kullanıcıya geç
COPY --chown=appuser:appgroup target/app.jar app.jar
USER 10001:10001

# 3. Bellek sınırını JVM'e bildir (K8s pod çökmesini engeller)
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75.0 -XX:+UseG1GC"
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Konteyneri root olmayan kullanıcıyla çalıştırırken port numarasını 1024'ün altında (örneğin port 80) seçmeyin; Linux geleneksel olarak 1024 altındaki portları sadece root'a tahsis eder. Her zaman 8080 gibi yüksek portlar seçin!

🇬🇧 WATCH OUT

Non-root users cannot bind to privileged ports below 1024 (e.g. port 80). Always configure Spring Boot on high ports like 8080.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Üretim ortamında ASLA konteynerleri root kullanıcısıyla çalıştırmayın.
- Distroless ve Alpine taban imajları saldırı yüzeyini ve dosya boyutunu küçültür.
- JVM `-XX:MaxRAMPercentage` ayarı bellek taşmalarını (OOMKilled) önler.

- NEVER execute containers as root in enterprise environments.
- Distroless and Alpine minimize the attack surface and download size.
- JVM container flags align Java heap limits with container cgroup limits.